

```

timescale 1ns / 1ps
`include "parameters.vh"
`default_nettype none

module data_mem_tb();

    reg clk;
    reg rst_n;
    reg i_stb;
    reg i_wr_en;
    reg [31:0] i_addr;
    reg [31:0] i_write_data;
    reg [4:0] i_amo_op;
    wire o_rd_ack;
    wire [31:0] o_read_data;

    // Instantiate the data memory module
    data_mem uut (
        .clk(clk),
        .rst_n(rst_n),
        .i_stb(i_stb),
        .i_wr_en(i_wr_en),
        .i_addr(i_addr),
        .i_write_data(i_write_data),
        .i_amo_op(i_amo_op),
        .o_rd_ack(o_rd_ack),
        .o_read_data(o_read_data)
    );

    // Clock generation
    always #5 clk = ~clk;

    initial begin
        // Initialize signals
        clk = 0;
        rst_n = 0;
        i_stb = 0;
        i_wr_en = 0;
        i_addr = 0;
        i_write_data = 0;
        i_amo_op = 0;

        // Reset sequence
        #10 rst_n = 1;
        #10;
    end
endmodule

```

```

// Write initial value to memory
i_stb = 1;
i_wr_en = 1;
i_addr = 32'h00000010;
i_write_data = 32'd30;
i_amo_op = 0; // Normal write
#10 i_stb = 0; i_wr_en = 0;
#10;

// Read back to check write
i_stb = 1;
i_wr_en = 0;
i_amo_op = 0;
#10;
$display("Read Data: %d (Expected: 30)", o_read_data);
#10 i_stb = 0;

// Perform AMO_ADD (30 + 5)
i_stb = 1;
i_wr_en = 1;
i_amo_op = `AMO_ADD;
i_write_data = 5;
#10;
$display("After AMO_ADD: %d (Expected: 35)", o_read_data);
#10 i_stb = 0; i_wr_en = 0;

// Perform AMO_MIN (35 vs. 25, should store 25)
i_stb = 1;
i_wr_en = 1;
i_amo_op = `AMO_MIN;
i_write_data = 25;
#10;
$display("After AMO_MIN: %d (Expected: 25)", o_read_data);
#10 i_stb = 0; i_wr_en = 0;

// Perform AMO_MAX (25 vs. 40, should store 40)
i_stb = 1;
i_wr_en = 1;
i_amo_op = `AMO_MAX;
i_write_data = 40;
#10;
$display("After AMO_MAX: %d (Expected: 40)", o_read_data);
#10 i_stb = 0; i_wr_en = 0;

// Perform Load-Reserved (LR)
i_stb = 1;

```

```

i_wr_en = 0;
i_amo_op = `AMO_LR;
#10;
$display("After AMO_LR: %d (Expected: 40)", o_read_data);
#10 i_stb = 0;

// Perform Store-Conditional (SC) - should succeed
i_stb = 1;
i_wr_en = 1;
i_amo_op = `AMO_SC;
i_write_data = 50;
#10;
$display("After AMO_SC (Success Case): %d (Expected: 50, SC should return
0)", o_read_data);
#10 i_stb = 0; i_wr_en = 0;

// External modification (to make SC fail)
uut.memory[32'h00000010] = 60;

// Perform Store-Conditional (SC) - should fail
i_stb = 1;
i_wr_en = 1;
i_amo_op = `AMO_SC;
i_write_data = 70;
#10;
$display("After AMO_SC (Fail Case): %d (Expected: 60, SC should return 1)",
o_read_data);
#10 i_stb = 0; i_wr_en = 0;

// End of simulation
$finish;
end

endmodule

```